

Capitolo 45 — PROT-016 — Deterministic State & Projection Contract

Pubblico	Lettori tecnici e non tecnici del WCM Process & Memory Book
Versione	FROZEN-45
Data master	2026-09-01
Stato	FROZEN
Source path	wcm/documentation/process-memory-book/ chapters/45_prot_016_deterministic_state_projection_contract.md
Source SHA	56f6532
Release	2026-09-04T09:44:14.164Z

GitHub main è la source of truth. Questo PDF è una release derivata: non costituisce approvazione né autorità.

PARTE VII — Il Libro dei Protocolli WCM Stato: FROZEN **Data:** 2026-09-01 **Scope:** WCM generale, domain-agnostic

45.0 Quando la stessa realtà non deve produrre due risposte diverse

Immaginiamo un sistema che deve rispondere a una domanda molto semplice:

| “Qual è lo stato corrente di questa attività?”

Se la risposta dipende ogni volta da come qualcuno interpreta una frase, da quale parola riconosce in un testo o da quale modello AI viene interpellato, due letture della stessa situazione possono produrre due risultati diversi.

Per alcune attività questa variabilità è tollerabile. Per altre è un problema strutturale.

Se dobbiamo riassumere una situazione complessa, proporre alternative o spiegare un concetto, l'interpretazione è utile. Se invece dobbiamo stabilire se un workflow è **ACTIVE**, **COMPLETED** o **WAITING_AUTHORITY**, oppure se dobbiamo proiettare lo stesso stato verso una vista destinata ad altri componenti, l'interpretazione libera diventa un rischio.

PROT-016 — Deterministic State & Projection Contract nasce per questo confine.

Il suo principio di fondo è:

| **Quando un fatto operativo può essere rappresentato in modo strutturato e verificabile, il sistema non deve reinventarne il significato ogni volta.**

Il protocollo non cerca di rendere deterministica tutta l'intelligenza del WCM. Cerca invece di rendere deterministici i passaggi in cui la variabilità non aggiunge valore: stato, identità, mapping, validazione e projection di dati strutturati.

45.1 Il problema che il protocollo risolve

Un sistema agentico può essere molto capace nel comprendere richieste ambigue, collegare informazioni e produrre sintesi. Ma queste stesse capacità diventano pericolose quando vengono usate per compiti che richiedono esattezza ripetibile.

Supponiamo che una fonte contenga un campo esplicito:

```
status = WAITING_AUTHORITY
```

Se questo campo esiste, non ha senso chiedere a un modello di “capire” se il testo sembra indicare un'attesa di approvazione. L'informazione è già presente in forma strutturata.

Il rischio di reinterpretarla è triplice.

Il primo rischio è la **variabilità**: lo stesso input può produrre output diversi.

Il secondo è la **deriva semantica**: una rappresentazione derivata può introdurre un significato che la fonte non possiede.

Il terzo è la **duplicazione incoerente**: più componenti possono scrivere la stessa realtà in modi diversi e competere tra loro.

PROT-016 riduce questi rischi imponendo contratti precisi nei punti in cui il significato può essere espresso con dati, enum, identificatori e mapping esatti.

45.2 Deterministico non significa “senza AI”

È importante chiarire subito un possibile equivoco.

Nel WCM, “deterministico” non significa che ogni decisione debba essere trasformata in una formula rigida. Significa che, **quando due esecuzioni ricevono lo stesso input strutturato valido, devono produrre lo stesso risultato previsto dal contratto.**

L'AI resta utile dove serve comprensione: interpretare un'intenzione, sintetizzare una situazione, valutare un significato, spiegare una conseguenza o proporre opzioni.

Il determinismo serve invece dove il significato è già stato deciso e rappresentato.

In forma semplice:

```
DA CAPIRE?  
→ cognition
```

```
GIÀ DEFINITO IN FORMA STRUTTURATA?  
→ contract + mapping deterministico
```

Il protocollo protegge proprio questo confine.

45.3 Il trigger: quando PROT-016 entra in gioco

PROT-016 si applica quando il WCM deve leggere, derivare, validare o proiettare stato operativo e informazioni strutturate che possiedono un contratto esplicito.

I trigger tipici sono:

- lettura dello stato di un workflow;
- derivazione di una vista di stato a partire dal runtime;
- costruzione di una projection destinata a una superficie di consultazione;
- trasferimento di dati strutturati verso un read-model;
- verifica di un gate rappresentato da campi canonici;
- replay della stessa informazione senza che debbano nascere duplicati;
- riconciliazione tra stato autorevole e vista derivata;
- migrazione di un renderer o di un transport che non deve cambiare l'identità logica degli oggetti.

Il protocollo non si attiva perché “c'è un file JSON” o perché “c'è del codice”. Si attiva perché il significato operativo è abbastanza strutturato da poter essere trattato con regole esatte.

45.4 L'input: il fatto strutturato viene prima della sua descrizione

Uno dei principi più importanti di PROT-016 è **Structured Before Text**.

In linguaggio semplice significa:

Se il sistema possiede già un campo strutturato che esprime un fatto, non deve ricostruire quello stesso fatto leggendo una frase.

Pensiamo a un modulo in cui esiste il campo:

```
approval_required = true
```

Una descrizione testuale può essere utile per spiegare il motivo dell'approvazione, ma non deve essere usata per decidere se l'approvazione sia richiesta quando il campo canonico lo dichiara già.

Questo evita che sinonimi, traduzioni, variazioni di wording o cambi di stile modifichino il comportamento operativo.

Il testo può cambiare. Il fatto strutturato, se non cambia il significato, resta lo stesso.

45.5 Execution Master: una fonte chiara per lo stato esecutivo

Per poter derivare lo stato in modo affidabile serve sapere dove si trova la fonte esecutiva autorevole.

Nel contratto corrente, i checkpoint strutturati del runtime rappresentano l'**Execution Master** dello stato operativo. Le viste che ne derivano — per esempio uno stato sintetico destinato a una persona o a un'altra superficie — non diventano automaticamente fonti equivalenti.

La relazione è:

```
EXECUTION MASTER
↓
VALIDAZIONE
↓
DERIVED STATE
↓
PROJECTION / VISTE
```

Questa gerarchia serve a evitare un errore comune: leggere una copia derivata, modificarla e poi trattarla come se avesse riscritto la realtà originaria.

Una vista può essere più leggibile. Non per questo possiede più authority.

45.6 Stati enumerati: scegliere tra valori noti, non interpretare l'ignoto

Uno stato operativo deve poter appartenere a un insieme definito.

La baseline di PROT-016 riconosce gli stati generali:

```
ACTIVE
INTERRUPTED_RESUMABLE
WAITING_AUTHORITY
BLOCKED
COMPLETED
CANCELLED
```

Il punto importante non è memorizzare i nomi. È capire la regola.

Se arriva un valore sconosciuto, il sistema non deve cercare quello “più simile”. Non deve decidere che **ALMOST_DONE** probabilmente significa **COMPLETED**, né che una variante testuale possa essere accettata perché semanticamente vicina.

Deve invece fallire in modo chiuso: **valore non riconosciuto, stato non valido, nessuna inferenza**.

Questo comportamento viene chiamato **fail closed**.

È l'opposto del “proviamo a indovinare”.

45.7 Un esempio quotidiano: il tabellone e l'orario ufficiale

Immaginiamo una stazione.

L'orario ufficiale contiene un treno con un identificatore preciso, una destinazione e uno stato aggiornato. Il tabellone è una rappresentazione destinata ai viaggiatori.

Se l'orario ufficiale dice che il treno è cancellato, il tabellone non dovrebbe leggere una nota testuale e “interpretare” se la cancellazione sembri probabile. Deve ricevere il valore previsto e mostrarlo secondo un mapping definito.

Se il tabellone viene sostituito con uno schermo più moderno, il treno non diventa un nuovo treno. Se la grafica cambia, l'identità del servizio non cambia. Se il sistema aggiorna due volte lo stesso evento, non dovrebbero comparire due cancellazioni diverse.

Questa metafora contiene già quattro idee centrali di PROT-016:

- fonte strutturata;
- mapping deterministico;
- identità logica stabile;
- idempotenza.

45.8 Identità logica stabile: cambiare rappresentazione senza cambiare oggetto

Un sistema distribuito usa identificatori per sapere se due rappresentazioni parlano della stessa cosa.

PROT-016 stabilisce che un cambio di wording, interfaccia, renderer o transport non autorizza automaticamente la creazione di una nuova identità.

Se un elemento è lo stesso oggetto logico, il suo identificatore deve restare stabile salvo una vera sostituzione o la nascita di una nuova entità.

Questo principio riguarda identificatori come:

```
document_id
need_id
item_id
event_id
workflow_instance_id
```

Per un lettore non tecnico, il concetto può essere ridotto a una domanda:

| *“Stiamo parlando della stessa cosa o di una cosa nuova?”*

Se è la stessa cosa, cambiare come viene mostrata non dovrebbe farle perdere la propria identità.

45.9 Fingerprint: riconoscere quando il contenuto logico non è cambiato

Per sapere se due input strutturati rappresentano lo stesso contenuto logico, il sistema può calcolare una **fingerprint**.

Una fingerprint è una sorta di impronta digitale del contenuto.

Ma per essere affidabile deve nascere da una rappresentazione canonica: chiavi ordinate, codifica definita, gestione coerente dei valori nulli, ordine stabile dove l'ordine non ha significato e assenza di elementi variabili come un timestamp aggiunto senza motivo al contenuto logico.

La baseline usa una hash **SHA-256** della rappresentazione canonica.

Il dettaglio matematico non è necessario per comprendere il principio:

| Se il significato strutturato non cambia, l'impronta deve restare uguale.

Questo permette al sistema di distinguere un vero delta da una semplice nuova esecuzione dello stesso input.

45.10 Idempotenza: ripetere senza duplicare

Un'operazione è **idempotente** quando ripeterla con lo stesso input non produce effetti aggiuntivi indesiderati.

È un concetto essenziale per heartbeat, retry, replay e sistemi distribuiti.

Supponiamo che una projection venga eseguita due volte perché la prima risposta di rete non è arrivata. Se l'input logico è lo stesso, la seconda esecuzione non dovrebbe creare un secondo documento, un secondo bisogno, un secondo evento o una falsa attività.

PROT-016 esprime questa idea così:

```
STESSO INPUT LOGICO
→ STESSA FINGERPRINT
→ NESSUN DUPLICATO LOGICO
```

L'idempotenza non significa che nulla possa mai cambiare. Significa che **la ripetizione dello stesso fatto non deve essere scambiata per un fatto nuovo.**

45.11 Projection: mostrare senza reinterpretare

Una projection è una vista derivata costruita per uno specifico uso.

Può servire a una persona, a un'interfaccia o a un read-model. Il suo compito è rendere disponibile una parte della realtà in una forma adatta al destinatario.

PROT-016 definisce il ruolo del projector in modo molto restrittivo:

```
LOAD STRUCTURED SOURCES
→ VALIDATE
→ MAP EXACTLY
→ FINGERPRINT
→ TRANSPORT
```

Il projector non decide il canone. Non decide l'authority. Non sceglie il prossimo passo del workflow. Non reinterpreta il significato operativo.

La sua forza sta proprio nel non “essere creativo”.

Se serve creatività o interpretazione, quel compito appartiene a un'altra parte del sistema.

45.12 Boundary Separation: ogni superficie possiede il proprio significato

Un altro rischio nasce quando lo stesso significato viene duplicato in più posti.

Se una superficie possiede già lo stato esecutivo, un'altra superficie non dovrebbe ricreare una seconda versione indipendente dello stesso stato senza un contratto esplicito.

PROT-016 chiama questo principio **Boundary Separation**.

In termini semplici:

ogni area deve sapere quale informazione possiede e quale invece riceve come derivazione.

La separazione riduce i conflitti del tipo:

- una vista dice **ACTIVE**;
- un'altra dice **COMPLETED**;
- una terza ricostruisce lo stato da una frase;
- nessuno sa quale delle tre prevalga.

Il protocollo evita questo scenario definendo ownership e mapping.

45.13 Single Writer: una realtà, un solo scrittore attivo per quel boundary

Se due componenti possono scrivere contemporaneamente la stessa projection, nasce una gara.

Uno scrive il valore nuovo. L'altro, partendo da una fotografia più vecchia, scrive subito dopo e lo sovrascrive. Il risultato finale può dipendere solo dall'ordine casuale con cui arrivano le scritture.

PROT-016 introduce quindi il principio di **Single Writer Ownership**:

```
one logical boundary  
→ one active writer
```

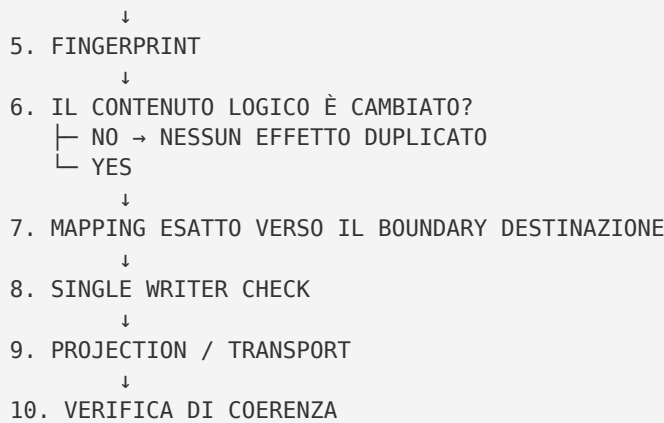
Non significa che un intero sistema debba avere un solo processo di scrittura. Significa che, per la stessa responsabilità logica, deve essere chiaro chi possiede il diritto operativo di produrre quella projection.

Questo riduce race condition e comportamenti “last write wins” non intenzionali.

45.14 Il flusso completo del protocollo

Possiamo ora ricostruire il flusso senza dettagli implementativi.

```
1. ARRIVA UN FATTO STRUTTURATO  
  ↓  
2. VERIFICA DEL CONTRATTO  
  ↓  
3. STATO / ENUM / IDENTITÀ VALIDI?  
  ├── NO → FAIL CLOSED  
  └── YES  
  ↓  
4. NORMALIZZAZIONE CANONICA
```



Il flusso è volutamente poco “intelligente”.

Qui la qualità non nasce dall'interpretazione, ma dalla ripetibilità.

45.15 I gate e i decision point

PROT-016 contiene diversi punti in cui il sistema deve fermarsi invece di improvvisare.

Gate 1 — Validità strutturale

Lo schema atteso è rispettato?

Se no, nessuna inferenza correttiva silenziosa.

Gate 2 — Stato conosciuto

Il valore appartiene all'insieme ammesso?

Se no, **INVALID_STATE** o equivalente contrattuale.

Gate 3 — Coerenza dei gate di authority

Se uno stato dichiara attesa di authority, il relativo contratto deve essere completo e canonico. Un gate malformato non viene trattato come “probabilmente valido”.

Gate 4 — Coerenza della projection

La projection rispetta il boundary previsto e non introduce campi che appartengono a un'altra responsabilità?

Se no, nessuna scrittura downstream.

Gate 5 — Ownership dello writer

Esiste un solo writer attivo per quel boundary?

Se no, esiste un conflitto di configurazione.

Questi gate non decidono che cosa “sarebbe meglio fare”. Verificano se il contratto è rispettato.

45.16 Gli output

Quando PROT-016 viene applicato correttamente, gli output non sono soltanto dati aggiornati.

Produce soprattutto proprietà operative:

- uno stato derivato coerente con l'Execution Master;
- una projection costruita da fonti strutturate validate;
- identità logiche preservate;
- fingerprint ripetibile;
- retry e replay senza duplicazioni logiche;
- una chiara ownership della scrittura;
- errori espliciti quando il contratto non può essere rispettato;
- possibilità di riconciliare viste stale senza reinventare lo stato.

Il risultato desiderato è quindi una catena in cui sia possibile spiegare **da quale input è nato un output e secondo quale mapping**.

45.17 Failure mode: come può rompersi il contratto

Il protocollo è importante soprattutto perché definisce cosa non deve accadere.

Un failure mode è uno stato sconosciuto che viene comunque interpretato.

Un altro è una projection che contiene campi appartenenti al boundary sbagliato.

Un altro ancora è un identificatore nuovo creato per lo stesso oggetto solo perché cambia l'interfaccia.

Oppure due writer attivi sulla stessa projection.

Oppure un path locale o temporaneo trasportato come se fosse un riferimento canonico condivisibile.

Oppure una vista derivata stale che viene trattata come fonte più autorevole del runtime.

In tutti questi casi il protocollo preferisce **fermare la propagazione** piuttosto che produrre una rappresentazione apparentemente plausibile ma strutturalmente incerta.

La regola è:

```
CONTRATTO NON VERIFICABILE  
→ NO SILENT GUESS  
→ NO DOWNSTREAM WRITE
```

45.18 Reconciliation: quando la vista resta indietro

Può accadere che l'Execution Master sia corretto ma una vista derivata sia stale.

PROT-016 non risolve il problema scegliendo la vista “più recente a occhio”. Il principio è invece:

```
MASTER VALIDO
+
VISTA STALE
→ RIGENERA LA VISTA DAL MASTER
```

Questo è il senso della riconciliazione deterministica.

La vista non viene corretta inventando una frase migliore. Viene riallineata ricostruendo il risultato previsto dal contratto a partire dalla fonte esecutiva valida.

Qui il protocollo si collega direttamente al processo di Deterministic State Reconciliation.

45.19 Relazioni con gli altri elementi WCM

PROT-016 non vive isolato.

È strettamente collegato a **PROC-011 – Deterministic State Reconciliation**, che usa il runtime strutturato per riallineare Derived State e viste esecutive.

Si collega a **PROC-005 – Agent-Ready Context Bootstrap**, perché il bootstrap operativo deve leggere prima lo stato strutturato quando disponibile e non affidarsi a una sintesi testuale stale.

Si collega a **PROT-009 – Contiguous Workflow Execution**, perché Resume Priority e next transition dipendono da uno stato esecutivo affidabile.

Si collega a **PROT-017 – Persistent Mutation Safety**, che disciplina la sicurezza delle mutazioni persistenti: un mapping deterministico corretto non elimina la necessità di controllare authority, target e precondizioni di una write.

Si collega inoltre alla separazione più ampia tra **Cognitive Core** e **Deterministic Core**. Il primo comprende e ragiona. Il secondo applica contratti meccanici dove il significato è già strutturato.

PROT-016 è uno dei punti in cui questa separazione diventa operativa.

45.20 Che cosa PROT-016 non autorizza

Il protocollo non attribuisce authority a un projector.

Non consente a una projection di cambiare il canone.

Non permette a un read-model di decidere il prossimo passo del workflow.

Non autorizza un componente deterministico a interpretare una decisione ambigua.

Non rende automaticamente sicura qualsiasi scrittura solo perché il payload è strutturato.

Non rende l'intero WCM deterministico.

E soprattutto non trasforma un campo ben formato in una decisione valida se manca l'autorità che dovrebbe averla prodotta.

La struttura riduce l'ambiguità. Non crea legittimità dal nulla.

45.21 Maturity e limiti

La baseline corrente di PROT-016 è attiva e possiede evidenze di field validation su implementazioni operative reali. Questo è un livello di maturità superiore a un semplice concept o POC.

Non significa però che ogni possibile sistema, workflow, renderer, transport o read-model sia già stato validato in ogni contesto.

Le proprietà già definite con forza sono il contratto, gli invarianti, i failure mode principali e diversi test di regressione. La generalizzazione continua a dipendere dalla corretta adozione dei boundary e dei contratti nei contesti in cui vengono applicati.

È quindi corretto dire:

| *PROT-016 è una baseline attiva con field validation significativa.*

Non sarebbe corretto dire:

| *PROT-016 dimostra che qualunque sistema agentico diventa automaticamente deterministico.*

Il protocollo rende deterministici **specifici confini strutturabili**, non la cognizione nel suo complesso.

45.22 La regola da ricordare

Se resta una sola idea di questo capitolo, deve essere questa:

| **Non chiedere all'intelligenza di reinterpretare ciò che il sistema può già rappresentare, validare e mappare in modo esatto.**

L'AI deve essere usata dove serve capire.

Il contratto deterministico deve essere usato dove serve ripetere lo stesso significato senza variarlo.

È questa separazione che permette al WCM di combinare capacità cognitiva e affidabilità operativa senza confonderle.

Source map del capitolo

Fonte primaria:

- [wcm/process-book/protocols/PROT-016_DETERMINISTIC_STATE_PROJECTION.md](#) — baseline canonica **ACTIVE / M3 FIELD VALIDATED**; il dettaglio contestuale delle evidenze di campo resta nella fonte tecnica e non viene riprodotto qui per mantenere il libro domain-agnostic.

Fonti collegate strettamente necessarie:

- [WCM_AGENT_START.md](#) — gerarchia esecutiva, Resume Priority, **Structured Before Text**, fail-closed e separazione tra cognition e componenti deterministici;
- [wcm/documentation/process-memory-book/BOOK_INDEX.md](#) — mapping editoriale CH45 → PROT-016 e principio dei due pass editoriali.

Maturity qualifier

Il capitolo descrive la baseline corrente e le invarianti attive del protocollo. La field validation documentata dimostra il comportamento in contesti operativi specifici, ma non costituisce prova di universalità per ogni possibile implementazione o dominio.